



Réunion 2

Clément Sinon

Table des matières

1	Introduction	3
1.1	Présentations personnelles	3
2	Programmation	4
2.1	Pair programming	4
2.2	Conventions de code	4
2.3	Dépôt GitLab	5
2.4	User stories	5
2.5	Début de l'itération 1	5
3	Divers	7
3.1	Tutorat	7
3.2	Serveur Discord	7
3.3	Sujets non planifiés	7

Chapitre 1

Introduction

1.1 Présentations personnelles

Les membres qui n'ont pas pu participer au tour de table de la première réunion seront invités à partager avec le groupe :

- ce qu'ils souhaitent apprendre,
- ce qui leur donne des difficultés,
- leurs attentes.

Chapitre 2

Programmation

2.1 Pair programming

XP prône le travail en binômes. L'un est le pilote et code, l'autre est le copilote et fait du *code review* en *live*.

Les binômes devraient être changés régulièrement afin de ne pas créer de “silo” de connaissance dans le code. Idéalement, d'ici la fin du projet, tout le monde aura travaillé au moins une fois avec chaque autre membre. Par conséquent, décider des binômes par affinités personnelles peut être jugé contraire à la philosophie d'XP. Une tournante prédéfinie ou un tirage aléatoire pourraient être envisagés.

Une difficulté que nous rencontrerons qui est d'ordinaire inexistante en entreprise est nos différences de disponibilités.

Le cours prévoit 11h de travail par semaine par étudiant. Ceci inclut le travail en tant que pilote et en tant que copilote.

Vote 2.1

Comment décider des binômes.

XP prône des changements de binômes fréquents pour mieux favoriser le partage d'information. Cependant, des changements trop fréquents empêcheraient les membres de s'habituer à leurs nouveaux binômes.

Vote 2.2

A quelle fréquence changer les binômes ?

Certains membres pourraient aussi embrasser un rôle de support. Ils pourraient être contactés en cas de difficulté par un binôme. Ces membres auraient des contraintes de disponibilité différentes des autres.

Vote 2.3

Création d'un rôle dédié au support.

2.2 Conventions de code

La propreté et l'homogénéité du code sont essentielles au bon fonctionnement de l'*extreme programming*. Il est donc impératif de définir des conventions.

Un socle d'XP est le *test-driven development*. Les tests unitaires ne servent pas à vérifier qu'une fonctionnalité fonctionne ; ils existent pour protéger les fonctionnalités d'un *refactoring* destructeur.

Les tests unitaires sont écrits avant les fonctionnalités qu'ils testent. Aucune fonctionnalité n'est implémentée si elle ne sert pas à valider un test.

Vote 2.4 Conventions de code

- anglais ou français,
- *snake case* ou *camel case*,
- indentation avec espaces ou tabulations,
- style des accolades,
- noms des *getters*, *setters* et *unit tests*,
- complétude de la *javadoc* (`@param`, `@return`, `@throws`, `@author`, `@version`),

Vote 2.5

Code-t-on des tests unitaires pour les constructeurs, *getters* et *setters* ?

Utiliser un même IDE pourrait simplifier l'entraide entre les membres. Mais certains pourraient être dérangés dans leurs habitudes par une telle standardisation.

Vote 2.6

Force-t-on les gens à utiliser un IDE spécifique ?

2.3 Dépôt GitLab

Un dépôt GitLab sera fourni pour le groupe par les assistants du cours.

Suivant la philosophie XP, le code sera constamment manipulé et réfactoré par tous les membres du groupe. Cela mènera inévitablement à du chaos. Afin de lutter contre cela, définir une structure des dossiers et fichiers pourrait s'avérer utile.

Tâche 2.1

Déterminer une structure d'organisation du code source.

Définir un format pour formuler les *commits* permettrait de mieux en naviguer l'historique.

Tâche 2.2

Définir un format pour les *commits*. Déterminer comment préciser les copilotes en co-auteurs.

2.4 User stories

Le projet est défini par une liste de *user stories* – essentiellement, de fonctionnalités – à implémenter. Une tentative de les organiser visuellement est fournie en [figure 2.1](#).

A priori les *stories* ne sont pas assignées à l'avance en XP. Elles sont réservées par un binôme en début de session de travail, et pour la durée de cette session. Cependant certains membres pourraient avoir des affinités particulières pour une *story* plutôt qu'une autre.

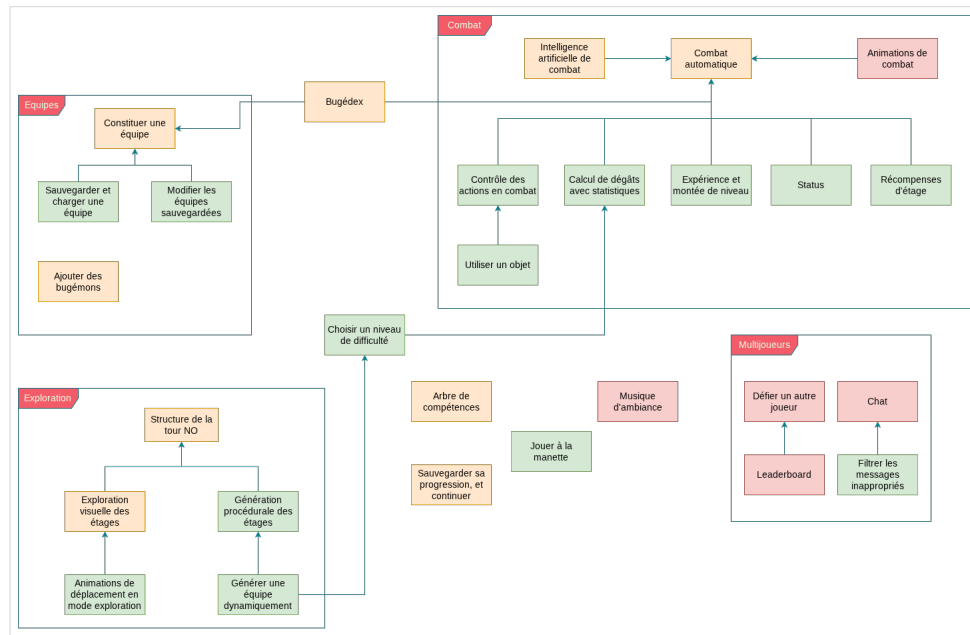
Vote 2.7

Comment attribuer les *stories* ?

2.5 Début de l'itération 1

Il est demandé pour le mardi 24 février d'estimer pour chaque *story* :

Figure 2.1 Plan des user stories



Son importance

Une priorité donnée à la *story*, de 1 à 3.

Son risque

La difficulté qu'elle représente, de 1 à 3.

Son temps

Le nombre d'heures de programmation qui devront lui être dédiées.

Vote 2.8

Comment évaluer les *stories*. Se les répartir ou faire ça en commun ?

Chapitre 3

Divers

3.1 Tutorat

Certains membres sont peu familiers avec git et GitLab, des outils qui seront centraux à ce projet. Il pourrait être judicieux d'organiser une session tutoriel pour ceux-ci.

Tâche 3.1

Tutorat de git, GitLab et GitHub Desktop.

3.2 Serveur Discord

Certaines suggestions pour le plan de réunion pourraient ne pas être faites par timidité. Un système de suggestions anonymes pourrait être envisagé.

Vote 3.1

Système de suggestions anonymes.

3.3 Sujets non planifiés

Les sujets non planifiés peuvent être abordés en fin de réunion.

Tout membre a le droit, sans devoir fournir de justificatif, de rejeter un sujet non planifié pour qu'il soit plutôt discuté à une prochaine réunion.